МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Инженерно-экономический факультет
Кафедра экономической информатики

Отчет
по лабораторной работе №3

на тему «Marshalling»
по курсу «Современные технологии серверной разработки»

Выполнил: Р. В. Тамашевский
472303

Проверила: Е. А. Мельникова

Минск 2026

Цель: приобрести навыки работы с парсерами данных: преобразованием объектов Java в XML и JSON и обратно.

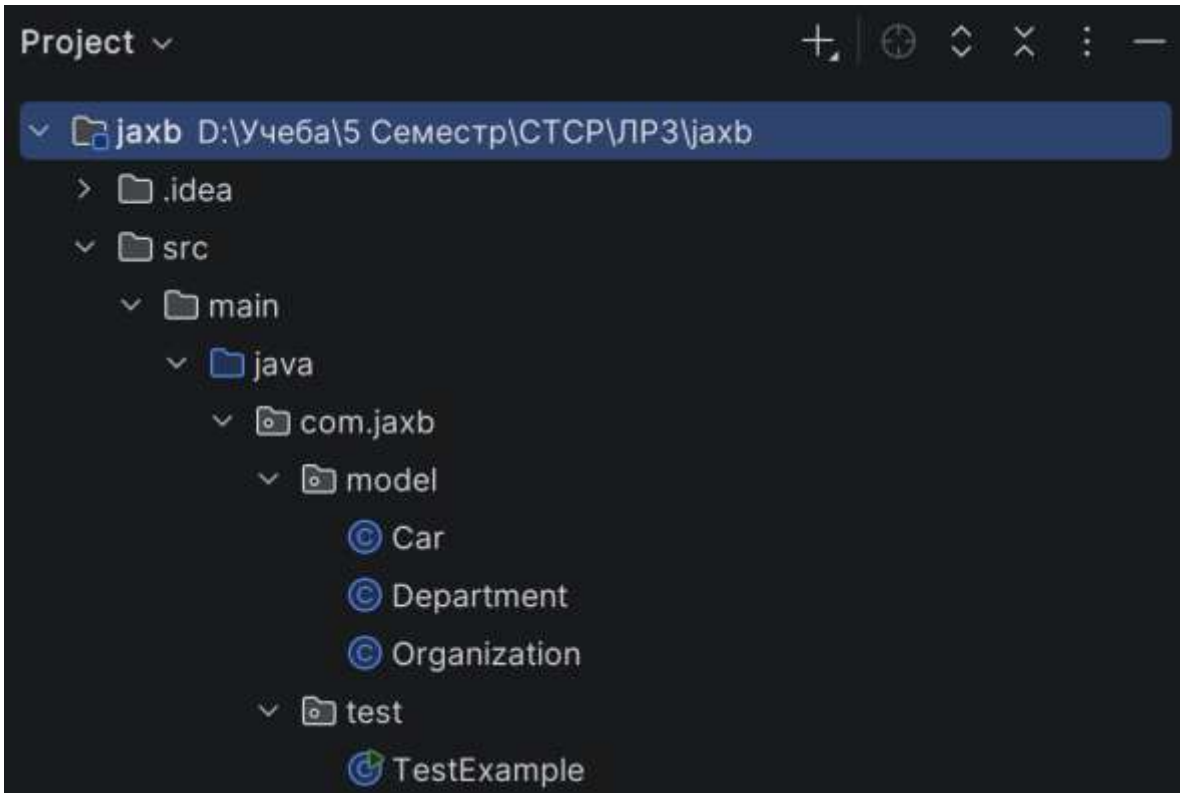
Задание: Изучить технологию JAXB и выполнить задание: добавить сущность Organization, содержащую список Department в качестве одного из полей, и реализовать запись и чтение организации из XML. Изучить библиотеку Gson и выполнить задание: записать массив объектов в JSON-файл, считать получившийся файл и вывести информацию на экран. Работа выполняется для предметной области варианта №12 «Автомобили» (марка, модель, год выпуска, цена, пробег): после чтения данных требуется найти автомобили, выпущенные после определённого года и с пробегом меньше заданного значения.

Ход выполнения работы:

Предметная область – сеть автосалонов. Организация (Organization) объединяет несколько автосалонов (Department), каждый из которых располагается в своём городе и содержит список автомобилей в наличии (Car). Автомобиль описывается маркой, моделью, годом выпуска, ценой и пробегом. Для обеих частей работы используется отбор автомобилей, выпущенных после 2018 года и с пробегом меньше 50 000 км.

1 JAXB

Создадим Maven-проект jaxb следующей структуры:



В pom-файл добавим зависимость jaxb-runtime – реализацию спецификации Jakarta XML Binding:

```
    <dependencies>
        <dependency>
            <groupId>org.glassfish.jaxb</groupId>
            <artifactId>jaxb-runtime</artifactId>
            <version>3.0.1</version>
        </dependency>
    </dependencies>
</project>
```

Создадим класс Car, описывающий автомобиль. Класс помечен аннотациями @XmlRootElement и @XmlAccessorType(XmlAccessType.FIELD), поэтому JAXB работает напрямую с полями класса. Поле carId помечено аннотацией @XmlAttribute и записывается в XML как атрибут id тега <car>, остальные поля – как дочерние элементы. Геттеры и сеттеры здесь и далее опущены.



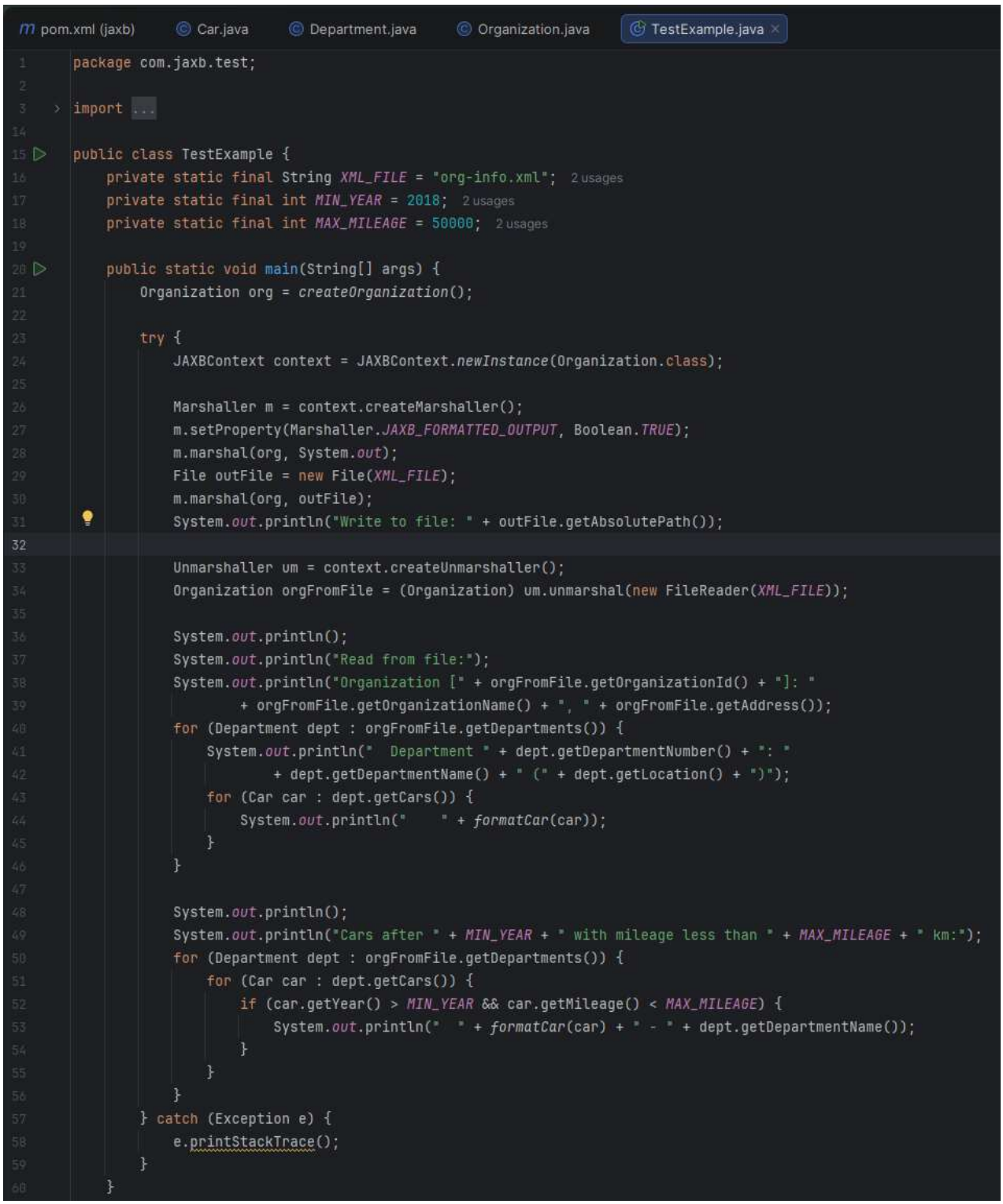
Класс Department описывает отдельный автосалон. Список автомобилей помечен аннотациями @XmlElementWrapper(name = "cars") и @XmlElement(name = "car"): все автомобили будут сгруппированы внутри тега <cars>, а каждый из них представлен тегом <car>.



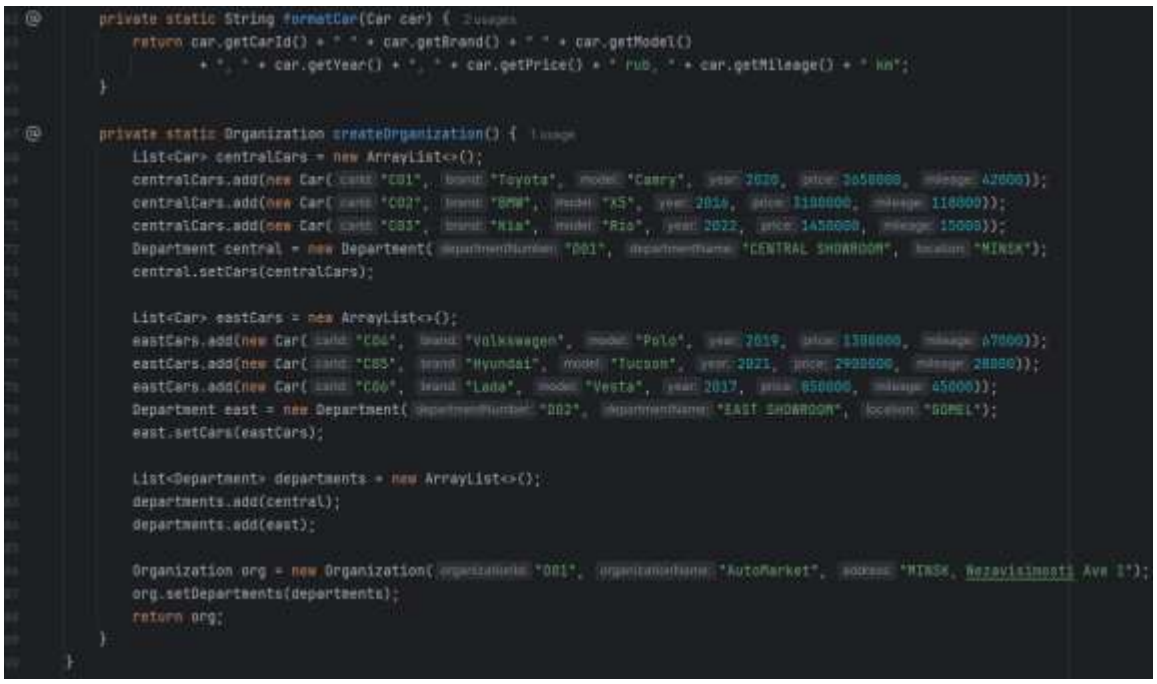
По заданию добавим сущность Organization, содержащую список Department в качестве одного из полей. Класс является корневым элементом XML (<organization>), идентификатор организации записывается атрибутом id, а список автосалонов оборачивается тегом <departments>.



В классе TestExample создадим JAXBContext для класса Organization. С помощью Marshaller с включённым свойством JAXB_FORMATTED_OUTPUT выполним маршалинг организации в консоль и в файл org-info.xml. Затем с помощью Unmarshaller выполним демаршалинг этого файла в объект Organization, выведем информацию об организации, её автосалонах и автомобилях, а также отберём автомобили, выпущенные после 2018 года и с пробегом меньше 50 000 км.



Вспомогательный метод formatCar() форматирует информацию об автомобиле, а метод createOrganization() создаёт организацию из двух автосалонов с шестью автомобилями:



```
    private static String formatCar(Car car) {
        return car.getCarId() + " " + car.getBrand() + " " + car.getModel()
                + ", " + car.getYear() + ", " + car.getPrice() + " rub, " + car.getMileage() + " km";
    }

    private static Organization createOrganization() {
        List<Car> centralCars = new ArrayList<>();
        centralCars.add(new Car( carId: "C01", brand: "Toyota", model: "Camry", year: 2020, price: 2650000, mileage: 42000));
        centralCars.add(new Car( carId: "C02", brand: "BMW", model: "X5", year: 2018, price: 3100000, mileage: 118000));
        centralCars.add(new Car( carId: "C03", brand: "Kia", model: "Rio", year: 2022, price: 1450000, mileage: 15000));
        Department central = new Department( departmentNumber: "D01", departmentName: "CENTRAL SHOWROOM", location: "MINSK");
        central.setCars(centralCars);

        List<Car> eastCars = new ArrayList<>();
        eastCars.add(new Car( carId: "C04", brand: "Volkswagen", model: "Polo", year: 2019, price: 1380000, mileage: 67000));
        eastCars.add(new Car( carId: "C05", brand: "Hyundai", model: "Tucson", year: 2021, price: 2920000, mileage: 28000));
        eastCars.add(new Car( carId: "C06", brand: "Lada", model: "Vesta", year: 2017, price: 850000, mileage: 45000));
        Department east = new Department( departmentNumber: "D02", departmentName: "EAST SHOWROOM", location: "GOMEL");
        east.setCars(eastCars);

        List<Department> departments = new ArrayList<>();
        departments.add(central);
        departments.add(east);

        Organization org = new Organization( organizationId: "O01", organizationName: "AutoMarket", address: "MINSK, Nezavisimosti Ave 1");
        org.setDepartments(departments);
        return org;
    }
}
```

В результате маршалинга в каталоге проекта появился файл org-info.xml:

pom.xml (jaxb) | Car.java | Department.java | Organization.java

```xml
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<organization id="001">
    <organizationName>AutoMarket</organizationName>
    <address>MINSK, Nezavisimosti Ave 1</address>
    <departments>
        <department>
            <departmentNumber>D01</departmentNumber>
            <departmentName>CENTRAL SHOWROOM</departmentName>
            <location>MINSK</location>
            <cars>
                <car id="C01">
                    <brand>Toyota</brand>
                    <model>Camry</model>
                    <year>2020</year>
                    <price>85000.0</price>
                    <mileage>42000</mileage>
                </car>
                <car id="C02">
                    <brand>BMW</brand>
                    <model>X5</model>
                    <year>2016</year>
                    <price>210000.0</price>
                    <mileage>118000</mileage>
                </car>
                <car id="C03">
                    <brand>Kia</brand>
                    <model>Rio</model>
                    <year>2022</year>
                    <price>55000.0</price>
                    <mileage>15000</mileage>
                </car>
            </cars>
        </department>
        <department>
            <departmentNumber>D02</departmentNumber>
            <departmentName>EAST SHOWROOM</departmentName>
            <location>GOMEL</location>
            <cars>
                <car id="C04">
                    <brand>Volkswagen</brand>
                    <model>Polo</model>
                    <year>2019</year>
                    <price>45000.0</price>
                    <mileage>67000</mileage>
                </car>
                <car id="C05">
                    <brand>Hyundai</brand>
                    <model>Tucson</model>
                    <year>2021</year>
                    <price>70000.0</price>
                    <mileage>28000</mileage>
                </car>
                <car id="C06">
                    <brand>Lada</brand>
                    <model>Vesta</model>
                    <year>2017</year>
                    <price>60000.0</price>
                    <mileage>45000</mileage>
                </car>
            </cars>
        </department>
    </departments>
</organization>
```

Результат демаршалинга файла и выборки автомобилей:

```
Write to file: D:\Учеба\5 Семестр\СТСР\ЛР3\jaxb\org-info.xml

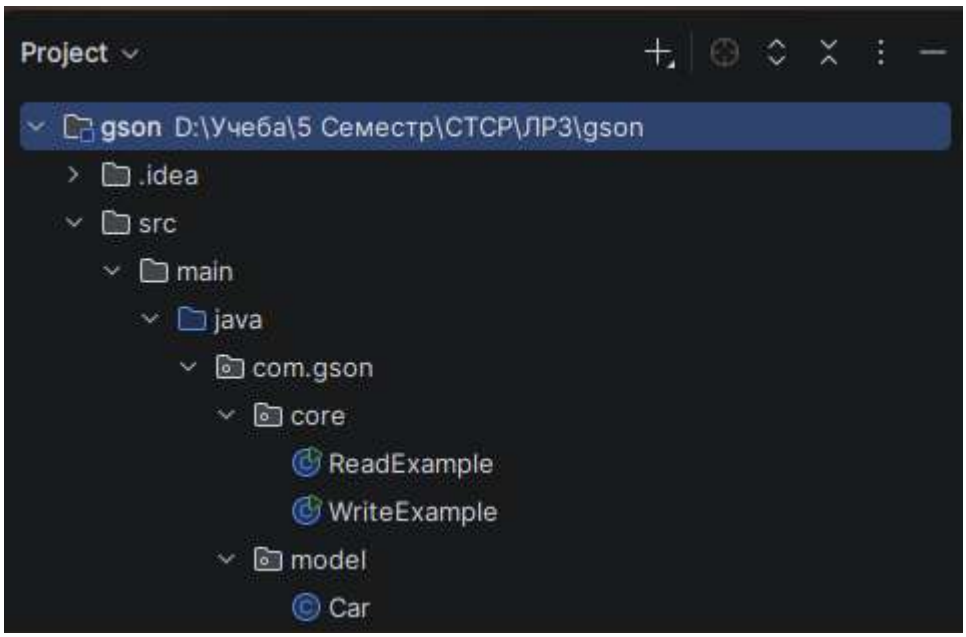
Read from file:
Organization [O01]: AutoMarket, MINSK, Nezavisimosti Ave 1
  Department D01: CENTRAL SHOWROOM (MINSK)
    C01 Toyota Camry, 2020, 85000.0 rub, 42000 km
    C02 BMW X5, 2016, 210000.0 rub, 118000 km
    C03 Kia Rio, 2022, 55000.0 rub, 15000 km
  Department D02: EAST SHOWROOM (GOMEL)
    C04 Volkswagen Polo, 2019, 45000.0 rub, 67000 km
    C05 Hyundai Tucson, 2021, 70000.0 rub, 28000 km
    C06 Lada Vesta, 2017, 60000.0 rub, 45000 km

Cars after 2018 with mileage less than 50000 km:
  C01 Toyota Camry, 2020, 85000.0 rub, 42000 km - CENTRAL SHOWROOM
  C03 Kia Rio, 2022, 55000.0 rub, 15000 km - CENTRAL SHOWROOM
  C05 Hyundai Tucson, 2021, 70000.0 rub, 28000 km - EAST SHOWROOM

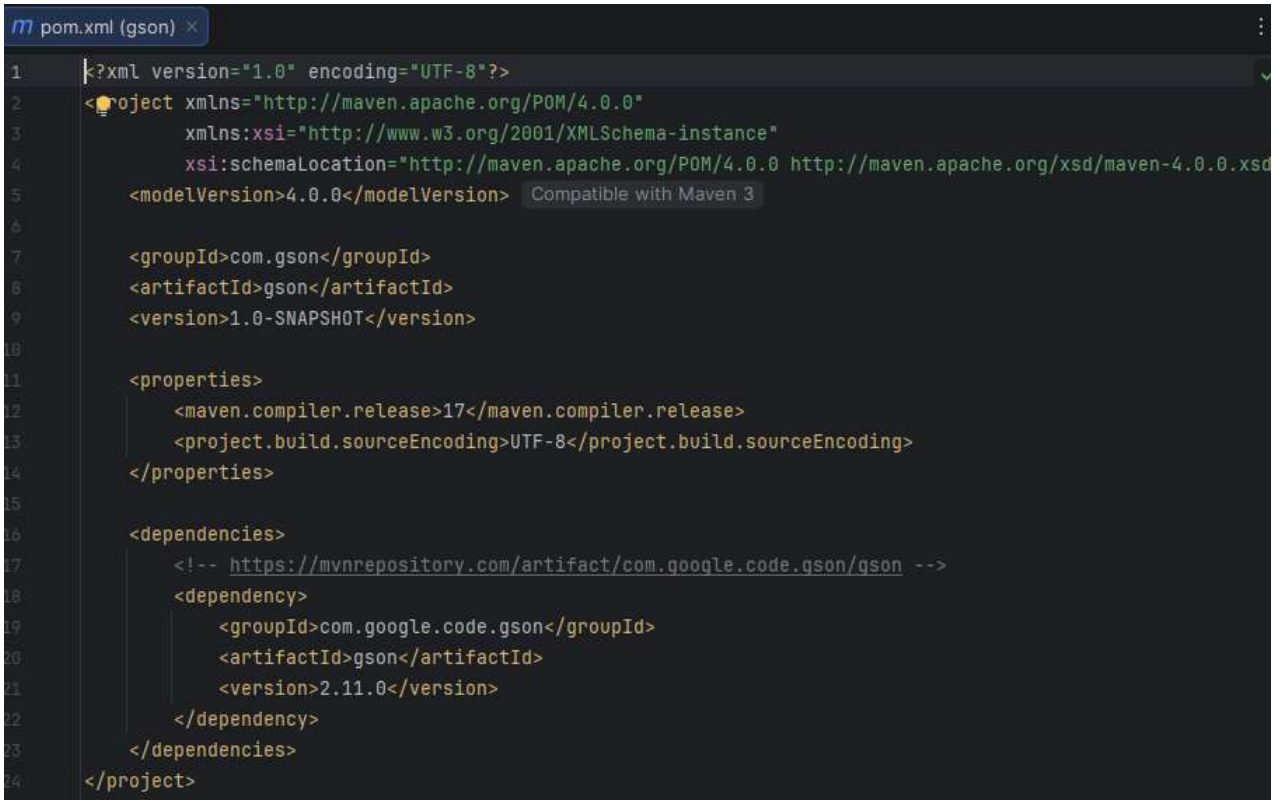
Process finished with exit code 0
```

2 Gson

Создадим Maven-проект gson следующей структуры:



В pom-файл добавим зависимость библиотеки Gson:



Создадим класс Car. Аннотации для Gson не требуются: библиотека получает доступ к полям объекта через рефлексию. Поскольку в классе нет полей из пакета java.time, подключать дополнительные адаптеры не нужно. Методы hashCode() и equals() опущены.

pom.xml (gson) | Car.java

```java
package com.gson.model;

import java.util.Objects;

public class Car {  16 usages
    private String brand;  7 usages
    private String model;  7 usages
    private int year;  7 usages
    private double price;  7 usages
    private int mileage;  7 usages

    public Car() {}  no usages

    public Car(String brand, String model, int year, double price, int mileage) {  5 usages
        this.brand = brand;
        this.model = model;
        this.year = year;
        this.price = price;
        this.mileage = mileage;
    }

    public String getBrand() { return brand; }

    public void setBrand(String brand) { this.brand = brand; }

    public String getModel() { return model; }

    public void setModel(String model) { this.model = model; }

    public int getYear() { return year; }

    public void setYear(int year) { this.year = year; }

    public double getPrice() { return price; }

    public void setPrice(double price) { this.price = price; }

    public int getMileage() { return mileage; }

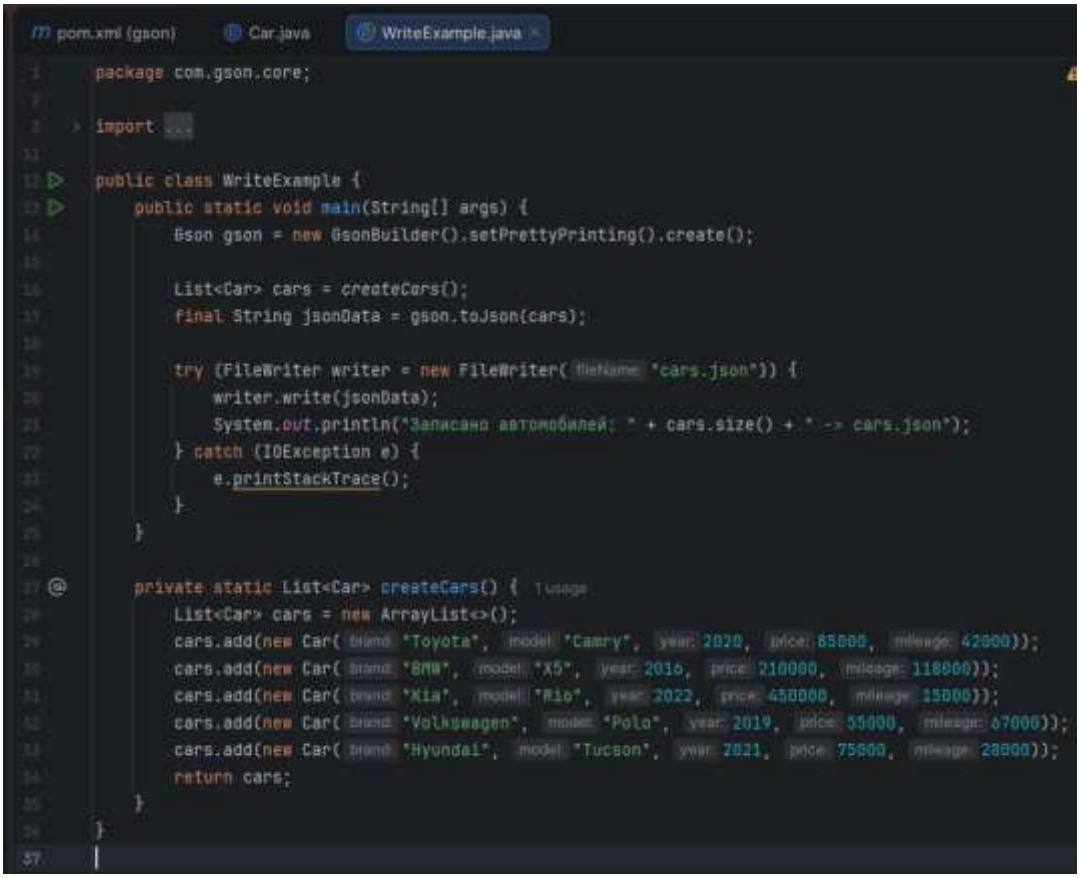
    public void setMileage(int mileage) { this.mileage = mileage; }

    @Override
    public String toString() {
        return "Car{" +
                "brand='" + brand + '\'' +
                ", model='" + model + '\'' +
                ", year=" + year +
                ", price=" + price +
                ", mileage=" + mileage +
                '}';
    }

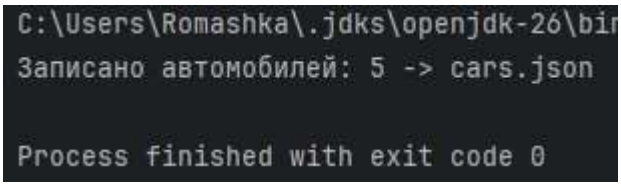
    @Override
    public int hashCode() { return Objects.hash(brand, model, year, price, mileage); }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o == null || getClass() != o.getClass()) return false;
        Car car = (Car) o;
        return year == car.year &&
                Double.compare(price, car.price) == 0 &&
                mileage == car.mileage &&
                Objects.equals(brand, car.brand) &&
                Objects.equals(model, car.model);
    }
}
```

В классе WriteExample создадим массив из пяти автомобилей. Объект Gson получим через GsonBuilder с настройкой setPrettyPrinting() для читаемого форматирования, преобразуем список в JSON-строку методом toJson() и запишем её в файл cars.json.



Результат выполнения WriteExample:



Содержимое полученного файла cars.json:

pom.xml (gson) | Car.java | WriteExample.java | cars.json



```
[
  {
    "brand": "Toyota",
    "model": "Camry",
    "year": 2020,
    "price": 85000.0,
    "mileage": 42000
  },
  {
    "brand": "BMW",
    "model": "X5",
    "year": 2016,
    "price": 210000.0,
    "mileage": 118000
  },
  {
    "brand": "Kia",
    "model": "Rio",
    "year": 2022,
    "price": 450000.0,
    "mileage": 15000
  },
  {
    "brand": "Volkswagen",
    "model": "Polo",
    "year": 2019,
    "price": 55000.0,
    "mileage": 67000
  },
  {
    "brand": "Hyundai",
    "model": "Tucson",
    "year": 2021,
    "price": 75000.0,
    "mileage": 28000
  }
]
```

В классе ReadExample считаем файл cars.json. Так как в файле хранится массив, при чтении через TypeToken<List<Car>> явно указывается тип данных – список автомобилей. После чтения выводим все автомобили и отбираем выпущенные после 2018 года с пробегом меньше 50 000 км.

pom.xml (gson) | Car.java | WriteExample.java | cars.json | ReadExample.java

```java
package com.gson.core;

import ...

public class ReadExample {
    private static final int MIN_YEAR = 2018;  2 usages
    private static final int MAX_MILEAGE = 50000;  2 usages

    public static void main(String[] args) {
        Gson gson = new Gson();

        Type listType = new TypeToken<List<Car>>() {}.getType();

        try (Reader reader = new FileReader( fileName: "cars.json")) {
            List<Car> cars = gson.fromJson(reader, listType);
            System.out.println("Прочитано автомобилей: " + cars.size());
            for (Car car : cars) {
                System.out.println(car);
            }

            System.out.println();
            System.out.println("Выпущены после " + MIN_YEAR + " года и пробег меньше " + MAX_MILEAGE + " км:");
            for (Car car : cars) {
                if (car.getYear() > MIN_YEAR && car.getMileage() < MAX_MILEAGE) {
                    System.out.println(car);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Результат выполнения ReadExample:

```
C:\Users\Romashka\.jdks\openjdk-26\bin\java.exe "-javaagent:D:\SOFT\IntelliJ ID
Прочитано автомобилей: 5
Car{brand='Toyota', model='Camry', year=2020, price=85000.0, mileage=42000}
Car{brand='BMW', model='X5', year=2016, price=210000.0, mileage=118000}
Car{brand='Kia', model='Rio', year=2022, price=450000.0, mileage=15000}
Car{brand='Volkswagen', model='Polo', year=2019, price=55000.0, mileage=67000}
Car{brand='Hyundai', model='Tucson', year=2021, price=75000.0, mileage=28000}

Выпущены после 2018 года и пробег меньше 50000 км:
Car{brand='Toyota', model='Camry', year=2020, price=85000.0, mileage=42000}
Car{brand='Kia', model='Rio', year=2022, price=450000.0, mileage=15000}
Car{brand='Hyundai', model='Tucson', year=2021, price=75000.0, mileage=28000}

Process finished with exit code 0
```

Вывод: В результате выполнения работы получены практические навыки работы с парсерами данных на языке Java. С помощью технологии JAXB реализованы маршалинг и демаршалинг иерархии объектов Organization →

Department → Car в XML-документ, изучены аннотации @XmlRootElement, @XmlAccessorType, @XmlAttribute, @XmlElement и @XmlElementWrapper. С помощью библиотеки Gson реализованы запись массива объектов в JSON-файл и его чтение с указанием типа через TypeToken, изучена настройка Gson через GsonBuilder. Для предметной области «Автомобили» выполнен отбор автомобилей, выпущенных после заданного года и с пробегом меньше заданного значения.